

MegaMek JFR Profiling Analysis

File: MegaMek_2026_06_21_200252.jfr

Duration: 1349 s (~22.5 min) Captured by: IntelliJ async-profiler GC: G1

Analyst: Claude Code Date: 2026-06-21

Bottom line (read this first)

- No memory leak in evidence. Heap recycles to a stable ~1 GB working set. (Caveat: only the active window is observed, and the dedicated leak-finder event was not enabled.)
 - The "allocation-heavy" instinct is right. The worst offender is a real, fixable bug: LaserWeapon.getModesCount rebuilds a stream on every call and churns ~32 GB. Fix that first.
 - FacingArc.valueOf (cache the values array) is a second easy, safe win.
 - Much of the rest of the load is rendering + GIF export + network + save, which may be an artifact of how this snapshot was recorded.
-

Step 1 - What is in the recording

The snapshot has the three things we need, plus one important gap.

Data	Events	Use
jdk.ObjectAllocationSample	146,572	What is allocated, and by which method
jdk.ExecutionSample	79,191	Where CPU time goes
jdk.G1HeapSummary jdk.GCHeapSummary	/ 1,862 / 897	Heap-after-GC trend (leak test)
jdk.OldObjectSample	0	MISSING - the dedicated leak-finder. Not enabled.

So allocation churn and CPU analysis are solid. Leak-vs-churn can be inferred from the heap trend, but a definitive leak hunt needs OldObjectSample (Step 6).

Step 3 - Leak vs churn: NO leak signal

(Stated before Step 2 because it is the main worry.)

Think of heap-after-GC like a sink that drains on each GC. A leak means the water line keeps rising no matter how often you drain. Healthy churn bounces up and down around a steady level (a sawtooth).

What the data shows:

- Heap-after-GC rose from 275 MB to ~1024 MB over the first ~130 s, then flattened.

- After warm-up it oscillates between a ~1024 MB floor and ~1230 MB, dropping back to the floor again and again - 86 reclaim events, right to the end of the GC window.
- The floor does NOT keep climbing. The "+256 MB first-vs-last-quarter" number is just warm-up (loading game, board, image caches), not a slow leak.

Plain terms: the program loads up to about a 1 GB working set, then holds steady and recycles memory normally. That is a sawtooth, not a leak.

CAVEAT: GC events only cover the first 633 s of the 1349 s recording. The back half had almost no GC activity (it lines up with 70,000 thread-sleeping samples - the app was largely idle). The no-leak verdict is solid for the active window only.

Step 2 - Allocation pressure: where the garbage comes from

Top object types by allocation weight (extrapolated bytes over the run - this is churn rate, not resident memory):

34 GB	int[]	12 GB	megamek.common.LosEffects
28 GB	megamek.common.board.BoardLocation	8 GB	java.awt.geom.AffineTransform
27 GB	java.lang.Object[]	8 GB	java.util.regex.Matcher
19 GB	byte[]	7 GB	com.squareup.gifencoder.Color
16 GB	java.util.stream.ReferencePipeline\$Head	7 GB	megamek.common.board.Coords
14 GB	java.util.stream.ReferencePipeline\$2	5 GB	megamek.common.ToHitData
13 GB	java.util.ArrayList\$Itr	4 GB	megamek.common.enums.FacingArc[]

Attributing each allocation to the closest method in our code:

32 GB	LaserWeapon.getModesCount	<-- #1, and it is basically a bug
28 GB	BoardLocation.of	<-- record churn in LOS/path loops
23 GB	ClientGUI\$4.gamePhaseChange	<-- UI listener rebuilds collections
20 GB	SanityInputFilter.checkInput	<-- regex on every network packet
18 GB	LosEffects.losForCoords	<-- line-of-sight math
17 GB	GifWriter.appendFrame	<-- GIF EXPORT was running
11 GB	NativeSerializationMarshaller.unmarshall	
9 GB	Game.getEntitiesVector	<-- defensive copy per call
4 GB	FacingArc.values()	<-- enum array cloned every call

Step 4 - CPU hotspots: a rendering + I/O workload, not bot math

Self-time leaf methods:

- 25% native code (.no_Java_frame) + 18% sun.java2d.loops.TransformHelper.Transform -> image scaling/drawing
- ~8% Deflater/Inflater -> gzip (save-game + network compression)
- gifencoder -> GIF encoding

Attributed to our code:

- ~18% board rendering (BoardView.drawHex 10.5% + BoardView.draw 5.8% + sprites)
- 6% GifWriter.appendFrame
- 11% network serialization (marshall + unmarshall)
- 1.3% GameManagerSaveHelper.saveGame
- Princess tactical math is small: toHitsImpossible 2%, losForCoords 0.3%

Plain terms: the CPU was busy drawing the board, encoding a GIF, sending network packets, and saving games - not running bot brains. A GIF was being recorded during this capture (GifWriter + gifencoder + heavy Java2D transforms + AffineTransform/Color allocations all point to it). That inflates both the allocation and CPU numbers a lot.

OPEN QUESTION: please confirm what you were actually doing during the capture. If GIF/board recording was on, most of the top CPU and a big chunk of the allocation is that feature, not gameplay.

Step 5 - Source-level findings and fixes

#1 (high-confidence bug): LaserWeapon.getModesCount - 32 GB churn

File: megamek/.../weapons/lasers/LaserWeapon.java:116

Current:

```
if (hasModes()) {
    synchronized (modes) {
        return (int) modes.stream()
            .filter(mode -> !mode.getName().startsWith("Pulse"))
            .count();
    }
}
```

This is the single biggest allocator in the whole recording. Every call builds a fresh stream pipeline - and those exact temp objects (ReferencePipeline\$Head, ReferencePipeline\$2, ReduceOps\$CountingSink, the lambda) sit at the top of the allocation-type list. The parent EquipmentType.getModesCount() is just modes.size() (O(1), zero allocation); this override is far more expensive. It is called constantly during UI mode display and to-hit checks.

Proposed fix - count with an indexed loop (no stream, no iterator on the Vector):

```
if (hasModes()) {
    synchronized (modes) {
        int count = 0;
        for (int i = 0; i < modes.size(); i++) {
            if (!modes.get(i).getName().startsWith("Pulse")) {
                count++;
            }
        }
        return count;
    }
}
```

```
}  
}
```

Even better if mode lists never change after construction: compute the pulse-filtered count once and cache it. The loop version is the safe, minimal change.

#2: FacingArc.valueOf(int) - 4 GB of cloned enum arrays

File: megamek/.../common/enums/FacingArc.java:177

```
public static FacingArc valueOf(int arcCode) {  
    for (FacingArc facingArc : FacingArc.values()) {    // values() clones every call  
        ...  
    }
```

Enum.values() returns a fresh array copy on every call - that is the 4 GB of FacingArc[].

Proposed fix - cache it once:

```
private static final FacingArc[] VALUES = values();  
// ...  
for (FacingArc facingArc : VALUES) { ... }
```

(A Map<Integer, FacingArc> lookup would be even faster, but the cached-array change is the low-risk one.)

#3: Game.getEntitiesVector() - 9 GB defensive copies

File: megamek/.../common/game/Game.java:1029

```
public synchronized List<Entity> getEntitiesVector() {  
    return Collections.unmodifiableList(inGameTWEntities());  
}
```

inGameTWEntities() rebuilds a list on every call, and this is called very frequently in hot loops. This is a bigger change (needs a cached list, invalidated on entity add/remove), so scope it as its own PR rather than fold it in. Flagging, not fixing yet.

#4: SanityInputFilter.checkInput - 20 GB, 15,458 samples (network path)

Runs on every deserialized network string and allocates java.util.regex.Matcher heavily (8 GB of Matchers in the type list). Worth checking whether Pattern objects are pre-compiled as static final and reused. Needs a read before a fix is proposed.

Step 5b - Git history: when each pattern was introduced

Found with git blame and the pickaxe (git log -S) against the megamek repo (remote MegaMek/megamek). Dates are commit dates.

Finding	Introduced	Commit	Author	Context
#1 LaserWeapon stream	2024-12-08	3f10043ef4	psikomonkie	Issue 6247: NPE with TacOps energy weapons on firing-mode change
#2 FacingArc.values() loop	2025-05-30	d72d34ced6	Luana Coppio	"wip: FacingArc enum" - born with the enum
#3 Game.getEntitiesVector copy	2023-01-07	3615e1d160	SJuliez	Long-standing; NOT a recent regression

Details:

- #1 is NOT a simple regression to revert. The version just before it returned `super.getModesCount() / 2` (cheap, but wrong for some laser configs). The stream was a deliberate correctness fix for a mode-count NPE; the per-call allocation was an unintended side effect. It was touched twice more after: `synchronized(modes)` added 2024-12-19 (5323db2bc9), laser-bay NPE handling 2024-12-29 (c8630a3b5c, what plain blame shows). The proposed loop fix keeps the correct Pulse-filtering behavior while dropping the allocation.
- #2 was present from the day the FacingArc enum was created. The cached-array fix is a pure optimization with no behavior change.
- #3 dates to 2023. The 2025-03-24 blame on the signature line is only a package move (`common/Game.java` -> `common/game/Game.java`). The cost is call frequency plus `inGameTWEntities()` rebuilding the list, not a code change.

On git bisect: not recommended here. Bisect needs an automated pass/fail test to label commits good/bad. This is an allocation regression - nothing crashes - so there is no natural test to bisect on without first writing an allocation benchmark. Blame + pickaxe already pinpoint the introducing commits with certainty, so bisect would add nothing. A more useful follow-up is an allocation GUARD test (assert `getModesCount` stays under an allocation budget) to stop a future silent regression.

Step 6 - What is inconclusive / what to capture next

1. A true leak cannot be ruled in or out from this file. The heap trend looks healthy (sawtooth, stable floor), but `jdk.OldObjectSample` - the event that actually names leaked objects - was off, and GC data covers only the first 633 s.

2. The capture is contaminated by GIF export + rendering. To measure bot/gameplay allocation, that feature is drowning it out.

To get a clean answer, capture again with:

- GIF/recording OFF, and ideally headless or with the board view minimized, so rendering does not dominate.
 - OldObjectSample enabled - in IntelliJ enable "Heap allocation / old object sampling", or run with `-XX:StartFlightRecording=settings=profile` (the profile config includes old-object sampling). That gives a real leak report.
 - A longer steady-state window - run bot turns for 10+ minutes WITHOUT going idle, so post-GC heap has time to show a real upward trend if one exists.
 - For a definitive leak check, a heap dump (`jmap -dump` or IntelliJ "Capture Memory Snapshot") at the end of a long run, opened in Eclipse MAT, beats JFR.
-

Suggested next action

Implement the `LaserWeapon.getModesCount` and `FacingArc.valueOf` fixes (with before/after diffs and a quick compile), and add an allocation guard test for `getModesCount` so it cannot silently regress again. `SanityInputFilter.checkInput` and `Game.inGameTWEntities()` can follow as separate, scoped changes.